

1.1. RTFM

Comment afficher la liste des fichiers dans un répertoire, y compris ceux qui commencent par un point (fichiers cachés)?

ls -a

Comment trouver le propriétaire d'un fichier ?

ls -l

A quoi sert la commande touch ?

Créer un fichier

A quoi sert l'utilitaire grep ?

Elle permet de faire des recherches puissantes

créez un répertoire TP1

mkdir TP01

Créez deux fichiers texte vides

touch file.txt file2.txt

ajoutez-y le texte suivant :

Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

Pour cela, utilisez deux méthodes :

en utilisant l'utilitaire echo et les caractères de redirection <, > ou |

echo "Lorem ipsum dolor sit amet,

consectetur adipiscing elit,

sed do eiusmod tempor incididunt ut labore et dolore magna aliqua." > file.txt

en utilisant l'éditeur de texte vi (ou, éventuellement, nano)

nano file2.txt

Ctrl + c

Ctrl + V

1.2.2. Propriétaires, groupes et droits

Afficher les deux dernières lignes du fichier

tail -n2 file.txt

Afficher les lignes du fichier qui contiennent dolor

grep "dolor" file2.txt

Afficher, parmi les deux dernières lignes du fichiers, celles qui contiennent labor.

tail -n2 file.txt | grep "labor"

1.2.2. Propriétaires, groupes et droits d'édérations

Créez un nouvel utilisateur, toto.

sudo adduser toto

Créez un nouveau groupe, newbies.

groupadd newbies

Attribuez toto au groupe newbies.

usermod -aG newbies toto

Changez les paramètres d'un de vos fichiers « lorem ipsum » pour que toto en soit propriétaire.

chown toto file.txt

Changez les paramètres de l'autre fichier pour qu'il appartienne au groupe newbies.

chgrp newbies file2.txt

Changez les permissions de vos fichiers pour que seul l'accès en lecture soit possible par le propriétaire, et en écriture par le groupe.

chmod 420 file.txt

chmod 420 file2.txt

Que fait la commande umask ?

La commande umask sous Linux sert à définir les permissions par défaut pour les nouveaux fichiers et dossiers créés par un utilisateur.

2. Les processus

ps sans aucun paramètre

ps

- Par défaut, **ps** affiche **les processus liés à ton terminal actuel et à ton utilisateur**.
- Typiquement, tu verras quelque chose comme :

PID	TTY	TIME	CMD
1234	pts/0	00:00:00	bash
5678	pts/0	00:00:00	ps

Explication :

- **PID** → identifiant du processus
- **TTY** → terminal associé
- **TIME** → temps CPU utilisé
- **CMD** → commande lancée

☞ Donc sans paramètres, **on ne voit pas tous les processus du système**, juste ceux de **ton shell**.

Options intéressantes de ps

Option	Description	Exemple
-e ou -A	Affiche tous les processus du système	ps -e
-f	Affiche les processus en format complet , avec PPID, UID, etc.	ps -ef
-u nom_utilisateur	Filtre les processus d'un utilisateur spécifique	ps -u jeremie
-x	Affiche aussi les processus sans terminal associé (services)	ps -x
--forest	Affiche les processus sous forme d'arbre (hiérarchie)	ps -ef --forest
-o	Permet de choisir les colonnes à afficher	ps -eo pid,cmd,%mem,%cpu

Principe de fonctionnement

Un processus sous Linux a toujours un **PID** (Process ID).

- Pour arrêter ou contrôler un processus, **kill** envoie un **signal** à ce PID.
- Chaque signal a un **numéro** et un **nom**.
- Le signal le plus courant pour tuer un processus est **SIGKILL** (numéro 9) ou **SIGTERM** (numéro 15).

Exemple basique

1. Trouver le PID d'un processus, par exemple gedit :

```
ps -e | grep gedit
```

Sortie :

```
1234 ? 00:00:01 gedit
```

- **PID = 1234**

1. Envoyer un signal **terminer normalement (SIGTERM)** :

kill 1234

1. Si le processus refuse de quitter, envoyer un signal **forcer (SIGKILL)** :

kill -9 1234

Paramètres intéressants

Paramètre	Description	Exemple
-l	Liste tous les signaux disponibles	kill -l
-s SIGNAL	Spécifie le signal à envoyer	kill -s SIGKILL 1234
-SIGNAL	Alternative pour envoyer un signal	kill -9 1234 (équivalent à SIGKILL)
PID	Identifiant du processus à cibler	kill 1234

Points importants

- **SIGTERM (15)** : demande au processus de se terminer proprement → peut être intercepté par le processus
- **SIGKILL (9)** : force l'arrêt immédiat → **impossible à intercepter**, le processus meurt directement
- **kill ne supprime pas le fichier ou le programme**, il arrête simplement le processus en cours

3. Installation de programmes

Installer l'éditeur Emacs

Selon la distribution Linux :

- **Debian / Ubuntu / Linux Mint** (utilise apt) :

sudo apt install emacs

- **RedHat / CentOS** (utilise yum) :

sudo yum install emacs

✓ **Explication** : install demande au gestionnaire de paquets de **télécharger et installer Emacs** avec toutes ses dépendances.

À quoi sert apt update ou yum update ?

- **apt update** :
 - Met à jour la **liste des paquets disponibles** à partir des repositories configurés.
 - Ne met pas à jour les logiciels eux-mêmes, juste les informations sur les versions disponibles.
- **yum update** :
 - Met à jour à la fois la **liste des paquets** et les **logiciels installés** (dépend de l'option utilisée).

Exemple :

sudo apt update

sudo apt upgrade

- **update** → rafraîchit la liste des paquets
- **upgrade** → installe les dernières versions disponibles

Lister les programmes installés

- **Debian / Ubuntu** :

dpkg -l

ou

apt list --installed

- **RedHat / CentOS** :

yum list installed

✓ Cela permet de voir tous les paquets logiciels présents sur la machine.

Pourquoi parle-t-on de repository ?

- Un **repository (ou dépôt)** est un **serveur qui contient des paquets logiciels**.
- Les gestionnaires de paquets (apt, yum) utilisent ces dépôts pour :
 - Télécharger les logiciels
 - Vérifier les mises à jour
 - Gérer les dépendances automatiquement

💡 **Exemple concret** : Ubuntu a ses repositories officiels (main, universe) qui contiennent tous les logiciels testés et prêts à installer.

4. Utilitaires réseaux

Les services réseaux utilisant évidemment les interfaces réseaux et les connectivités disponibles, vous serez fréquemment amenés à examiner la configuration IP d'une machine et à analyser son fonctionnement.

Prenez dès à présent l'habitude de représenter la situation courante sous forme d'un schéma réseau en préalable à toute opération de configuration ou de debugging.

Utilisez la commande `ip addr` pour afficher les informations de configuration des interfaces de la machine. Expliquez chaque élément de la réponse.

ip addr

on obtient la liste des interfaces réseau et leurs paramètres.

Prenons un exemple de sortie typique (simplifiée) :

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever

2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
default qlen 1000
    link/ether 00:0c:29:4d:9b:2e brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.50/24 brd 192.168.1.255 scope global dynamic ens33
        valid_lft 86348sec preferred_lft 86348sec
    inet6 fe80::20c:29ff:fe4d:9b2e/64 scope link
        valid_lft forever preferred_lft forever
```

Explication ligne par ligne

1: lo: <LOOPBACK,...>

- **1:** numéro d'interface attribué par le noyau.

- **lo** : nom de l'interface (loopback). Sert pour que la machine communique avec elle-même.
- **<LOOPBACK,UP,LOWER_UP>** : indicateurs de l'état de l'interface :
 - **LOOPBACK** : c'est une interface spéciale de test.
 - **UP** : l'interface est activée.
 - **LOWER_UP** : liaison physique/logique active.
- **mtu 65536** : taille max d'un paquet (Maximum Transmission Unit).
- **qdisc noqueue** : algorithme de gestion de file d'attente des paquets (ici pas de file).

Ensuite :

- **link/loopback 00:... →** adresse MAC fictive.
- **inet 127.0.0.1/8 →** adresse IPv4 de la boucle locale (localhost).
- **inet6 ::1/128 →** adresse IPv6 de la boucle locale.

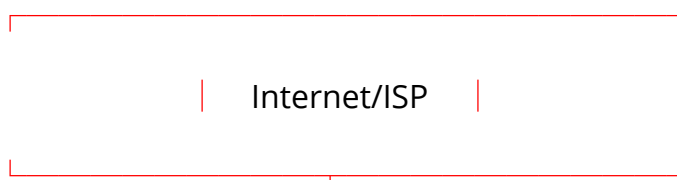
2: ens33: <BROADCAST,...>

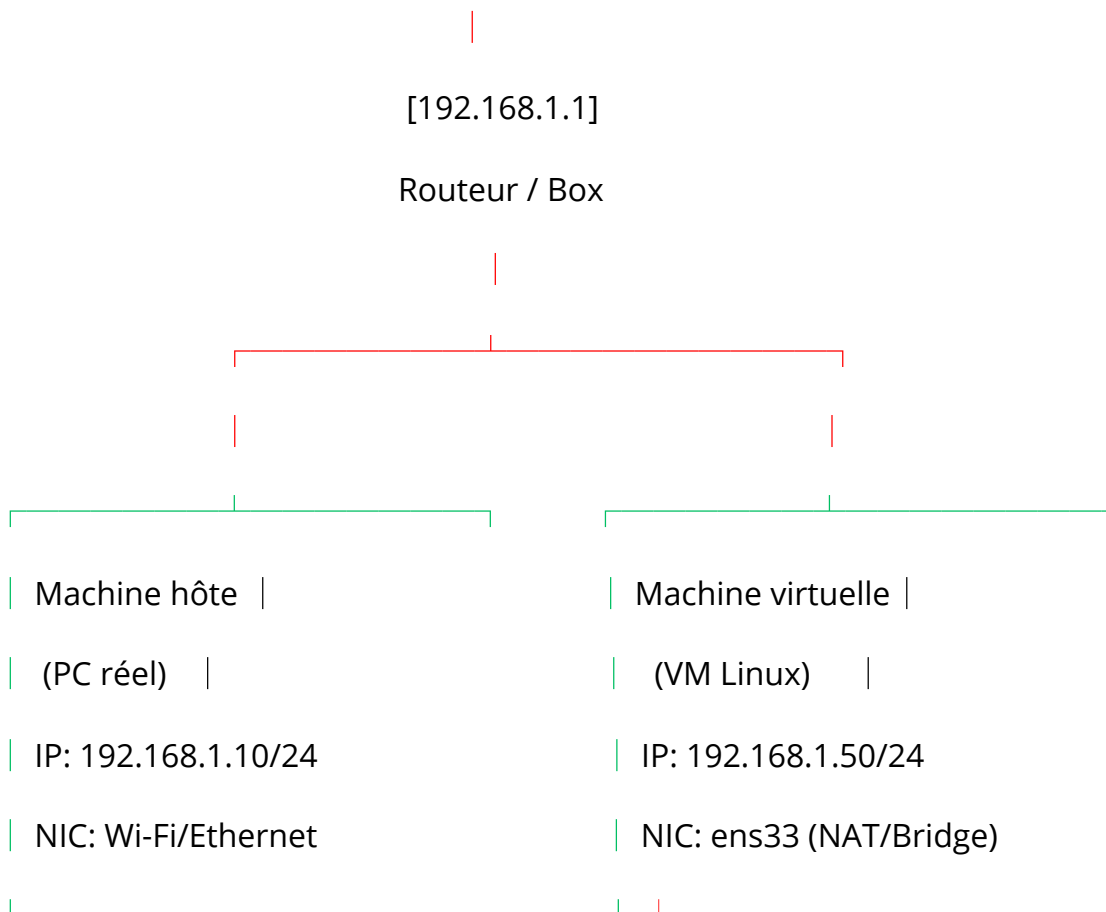
- **2** : numéro d'interface.
- **ens33** : nom de la carte réseau (dépend de la VM/OS).
- **<BROADCAST,MULTICAST,UP,LOWER_UP>** :
 - **BROADCAST** : peut envoyer des diffusions.
 - **MULTICAST** : supporte le multicast.
 - **UP** : interface activée.
 - **LOWER_UP** : câble/logique reliée.
- **mtu 1500** : taille max standard Ethernet.
- **link/ether 00:0c:29:4d:9b:2e** : adresse MAC.
- **inet 192.168.1.50/24** :
 - **Adresse IPv4 de la machine** : 192.168.1.50.
 - **Masque /24** → 255.255.255.0 → sous-réseau 192.168.1.0/24.
 - **brd 192.168.1.255** : adresse de broadcast du sous-réseau.
 - **scope global** : adresse routable localement.
 - **dynamic** : attribuée par DHCP.
- **inet6 fe80::20c:29ff:fe4d:9b2e/64** :
 - Adresse IPv6 link-local (commence par fe80::).
 - Automatiquement attribuée, sert pour la comm. locale.

Faites un schéma représentant votre machine virtuelle, votre machine hôte et leurs interfaces respectives. Indiquez les subnets IP correspondant à chaque interface.

Schéma machine hôte / machine virtuelle

Voici une représentation simplifiée :





- **Machine hôte** : ton PC, connecté au réseau local (LAN).
- **Machine virtuelle** : créée via VirtualBox/VMware, elle a sa propre carte réseau virtuelle (souvent en mode **Bridge** ou **NAT**).
- Les deux partagent le **même subnet 192.168.1.0/24** si c'est en mode *Bridge*.
- En mode **NAT**, la VM a une adresse privée (ex. 10.0.2.15/24), et c'est l'hôte qui fait la traduction vers le LAN.

A quoi sert l'utilitaire ping ?

ping est un outil réseau de diagnostic.

Il sert à tester la connectivité entre deux machines (hôte et cible).

Il envoie un petit message de test et attend la réponse.

Si la cible répond, on sait que :

- elle est joignable sur le réseau,
- il y a une connectivité IP entre les deux.

Expliquez en quelques mots, et utilisez-le avec la machine de votre voisin comme cible.

Si ton voisin a l'IP 192.168.1.20, tu tapes dans ta VM ou ton hôte :

ping 192.168.1.20

Sortie typique :

PING 192.168.1.20 (192.168.1.20) 56(84) bytes of data.

64 bytes from 192.168.1.20: icmp_seq=1 ttl=64 time=0.543 ms

64 bytes from 192.168.1.20: icmp_seq=2 ttl=64 time=0.478 ms

- icmp_seq = numéro du message envoyé.
- ttl = "time to live", nombre de sauts max dans le réseau.
- time = temps de réponse (latence).

Quelles sont les options de ping ?

Les options peuvent varier légèrement selon Linux, mais en voici les plus utiles :

Option	Description
-c <n>	Envoie un nombre précis de paquets (ex: ping -c 4 192.168.1.20).
-i <sec>	Intervalle entre deux pings (par défaut 1 sec).
-s <taille>	Taille du paquet ICMP envoyé.
-t <ttl>	Fixer la valeur TTL (durée de vie des paquets).
-q	Mode silencieux, affiche juste un résumé à la fin.
-f	Flood ping (envoie très vite plein de paquets, à utiliser avec prudence !).

Si vous connaissez déjà Wireshark, utilisez-le pour observer le trafic généré par cet outil.

Qu'est-ce que Wireshark ?

- C'est un **logiciel gratuit** qui permet de voir tout le trafic réseau qui passe par ta carte réseau.
- On l'appelle un **analyseur de paquets** (packet sniffer).

- Il affiche en temps réel tous les paquets envoyés/reçus par ta machine.

👉 Imagine un **scanner** qui regarde toutes les “lettres” (paquets) qui circulent dans le réseau et te les montre en détail.

À quoi ça sert ?

- Vérifier si deux machines communiquent (exemple : vérifier si ping marche).
- Comprendre quel protocole est utilisé (HTTP, DNS, ICMP, etc.).
- Déboguer des problèmes de réseau.
- Étudier le fonctionnement des protocoles réseau.

Exemple avec ping

Si tu lances Wireshark et que tu fais un ping 192.168.1.20, tu verras défiler des lignes comme :

- ICMP Echo Request (c'est ton “Coucou, tu es là ?”).
- ICMP Echo Reply (la réponse : “Oui je suis là”).

Chaque ligne est un paquet réseau. Tu peux cliquer dessus pour voir les détails :

- Adresse source et destination (IP, MAC).
- Protocole utilisé (ICMP dans ce cas).
- Contenu du message.

Résumé très simple

- ping = test de connexion avec des petits messages ICMP.
- **Wireshark** = logiciel pour *voir* ces messages passer sur le réseau.
- Tu y verrais des paires de messages :
 - **Echo Request** (envoi),
 - **Echo Reply** (réponse).

De quel protocole s'agit-il ?

- **Ping utilise ICMP** (*Internet Control Message Protocol*).
- ICMP est transporté **dans IP**.
- Donc **ping = ICMP → IP → Ethernet/Wi-Fi**.

Quelles sont les couches/protocoles sur lesquelles il repose ?

Protocole principal utilisé

- **ping** repose sur **ICMP** (*Internet Control Message Protocol*).
- ICMP n'utilise pas TCP ou UDP → il est directement transporté **dans IP**.

Couches du modèle OSI (simplifié)

- **Couche 7 – Application :**
 - Pas vraiment de protocole applicatif (ping est juste un outil).
- **Couche 4 – Transport :**
 - Pas utilisé (ni TCP, ni UDP).
- **Couche 3 – Réseau :**
 - **IP** (Internet Protocol) transporte ICMP.
 - L'adresse IP source et destination sont utilisées.
- **Couche 2 – Liaison de données :**
 - **Ethernet** (si câble) ou **Wi-Fi** (si sans fil).
 - Utilise les adresses MAC pour acheminer localement.
 - ARP peut intervenir pour résoudre l'adresse IP en adresse MAC.
- **Couche 1 – Physique :**
 - Transmission des bits par câble (cuivre/fibre) ou ondes radio (Wi-Fi).

👉 En résumé :

Le **ping** utilise **ICMP**, qui repose sur **IP**, lequel est transmis via **Ethernet/Wi-Fi** sur le support physique.

La commande **arp** vous permet de consulter la table ARP de la machine.

Qu'est ce que le protocole ARP ?

- **ARP = Address Resolution Protocol.**

A quoi sert-il ?

- Il sert à **traduire une adresse IP en adresse MAC.**
- Exemple : tu veux envoyer un paquet à 192.168.1.20. Ton PC sait que c'est dans le même sous-réseau (192.168.1.0/24), donc il a besoin de l'adresse **MAC** pour construire une trame Ethernet.
- Il envoie alors une requête **ARP broadcast** :
- "Qui a 192.168.1.20 ? Donne-moi ton adresse MAC."
- La machine 192.168.1.20 répond :
- "C'est moi, ma MAC est 08:00:27:aa:bb:cc."

En quoi consulter cette table peut vous être utile pour le troubleshooting réseau ?

Quand tu tapes :

arp -n

ou

ip neigh

tu vois la table ARP : correspondance entre adresses IP et MAC que ta machine connaît.

Exemple :

```
192.168.1.1 08:00:27:1a:2b:3c
```

```
192.168.1.20 08:00:27:aa:bb:cc
```

Utile pour le troubleshooting

- Si une IP n'apparaît pas → peut-être que la machine est éteinte ou inaccessible.
- Si une IP a la mauvaise MAC → conflit d'IP sur le réseau.
- Si la table est vide → problème de communication locale.

En résumé : la table ARP permet de vérifier que ton PC connaît bien les voisins IP de ton réseau local.

Même question avec la commande route.

La commande **route** ou **ip route** montre la **table de routage** de ta machine.

Elle indique **comment atteindre chaque réseau**.

Chaque ligne associe :

- un **réseau cible** (ex. 192.168.1.0/24),
- une **passerelle (gateway)**,
- une **interface réseau** (ex. eth0, wlan0),
- une **métrique** (priorité).

Exemple de sortie

```
default via 192.168.1.1 dev ens33
```

```
192.168.1.0/24 dev ens33 proto kernel scope link src 192.168.1.50
```

- **default via 192.168.1.1** → si je veux contacter une machine hors de mon LAN, j'envoie tout au routeur 192.168.1.1.
- **192.168.1.0/24 dev ens33** → pour toutes les IP de mon réseau local, j'utilise directement l'interface ens33.

Utile pour le troubleshooting

- Si la **route par défaut (default gateway)** est absente → ta machine ne peut pas sortir sur Internet.
- Si une route est mauvaise (ex. mauvaise passerelle) → perte de connectivité.
- Si plusieurs routes existent pour le même réseau → tu peux avoir un routage ambigu ou instable.

En résumé : la table de routage permet de vérifier que ta machine sait comment atteindre chaque destination.

A quoi sert l'utilitaire traceroute ?

C'est un utilitaire qui permet de voir **le chemin qu'empruntent les paquets sur le réseau** pour aller d'une machine A à une destination B (souvent un serveur Internet).

Il affiche **tous les routeurs intermédiaires (hops)** traversés.

On obtient :

- les adresses IP et parfois les noms des routeurs,
- le temps de réponse (latence) de chaque saut

👉 **En résumé :** ping teste la connectivité, traceroute montre le chemin.

Utilisez-le pour découvrir un maximum d'information sur le réseau dans lequel vous vous trouvez.

Le principe repose sur le champ **TTL (Time To Live)** dans l'entête IP :

1. Traceroute envoie un paquet avec **TTL = 1**.
 - Le premier routeur décrémente le TTL à 0 → il jette le paquet et renvoie un message **ICMP Time Exceeded**.
 - On sait donc qui est le premier routeur.
2. Traceroute envoie un paquet avec **TTL = 2**.
 - Le deuxième routeur renvoie un ICMP Time Exceeded.
3. Et ainsi de suite... jusqu'à atteindre la destination.

Quand la destination finale est atteinte, elle répond par un message ICMP différent (port unreachable si UDP, ou echo si ICMP).

Si vous savez le faire, observez le trafic généré avec Wireshark.

Si tu fais un traceroute vers 8.8.8.8 par exemple :

- Tu verras des paquets **UDP (Linux)** ou **ICMP (Windows)** partir avec TTL croissant.

- Et des réponses **ICMP Time Exceeded** revenir de chaque routeur intermédiaire.

Exemple de sortie (Linux)

traceroute 8.8.8.8

Résultat typique :

1	192.168.1.1 (192.168.1.1)	2.123 ms	1.987 ms	2.045 ms
2	10.12.0.1 (10.12.0.1)	5.123 ms	5.543 ms	5.654 ms
3	203.0.113.1 (203.0.113.1)	20.234 ms	20.112 ms	19.876 ms
...				
12	8.8.8.8 (8.8.8.8)	35.987 ms	36.001 ms	35.912 ms

Chaque ligne = un **saut** (hop).

3 temps = 3 mesures successives (latence en millisecondes).

De quels protocoles s'agit-il ?

Sous Linux :

- Par défaut, traceroute utilise **UDP** (envoi des datagrammes UDP vers un port élevé, ex. 33434).
- Chaque routeur envoie une réponse **ICMP Time Exceeded**.
- La machine de destination envoie un **ICMP Port Unreachable** (puisque le port UDP est "faux").
- Donc : **Traceroute = UDP + ICMP**.

Sous Windows (tracert) :

- Il utilise directement **ICMP Echo Request** (comme ping).
- Chaque routeur renvoie **ICMP Time Exceeded**.
- Donc : **Tracert = ICMP seulement**.

traceroute tourne-t-il au dessus de TCP ou d'UDP ?

UDP

Comparez sous Linux et sous Windows (outil tracertr).

Systeme	Commande	Protocole utilisé	Réponses attendues
Linux	traceroute	UDP (vers ports élevés) + ICMP	ICMP Time Exceeded + ICMP Port Unreachable
Windows	tracert	ICMP Echo Request	ICMP Time Exceeded + Echo Reply

⚠ [IMPORTANT]

A quoi sert l'utilitaire netstat ?

netstat (*network statistics*) est un outil qui affiche l'état des connexions réseau et des sockets de la machine.

Il permet de voir :

- les connexions **actives** (TCP/UDP),
- les **ports ouverts** en écoute,
- les **interfaces réseau** et leurs statistiques,
- les **routes** connues par la machine.

👉 **C'est un outil de diagnostic** : "quels services tournent sur ma machine et sur quels ports ?"

Quels sont les effets de la commande **netstat -lnptu** ?

affiche tous les **ports TCP/UDP ouverts en écoute**, avec leur **adresse IP**, leur **port**, et le **processus** qui les utilise.

Utilisez la manpage pour déterminer le rôle de chaque option.

Décortiquons les options une par une (via man netstat) :

- **-l (listening)**
 - Montre uniquement les **sockets en écoute** (c'est-à-dire les ports ouverts par des services, prêts à recevoir des connexions).

- **-n (numeric)**
→ Affiche les adresses et ports sous forme **numérique** (IP/port) plutôt que de résoudre en noms de domaine ou services.
Exemple : affiche 0.0.0.0:80 au lieu de *:http.
- **-p (program)**
→ Montre quel **processus** (PID et nom) utilise chaque socket.
⚠ Nécessite souvent les privilèges root (sudo).
- **-t (TCP)**
→ Affiche uniquement les sockets **TCP**.
- **-u (UDP)**
→ Affiche uniquement les sockets **UDP**.

Exemple de sortie typique

Commande :

sudo netstat -lnptu

Résultat :

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State	PID/Program name
tcp	0	0	0.0.0.0:22	0.0.0.0:*	LISTEN	784/sshd
tcp6	0	0	:::80	:::*	LISTEN	912/apache2
udp	0	0	0.0.0.0:68	0.0.0.0:*		635/dhclient

- **tcp** → protocole.
- **0.0.0.0:22** → service SSH écoute sur toutes les interfaces, port 22.
- **:::80** → service Apache écoute en IPv6 sur port 80.
- **udp 0.0.0.0:68** → client DHCP en attente de réponses.
- **PID/Program name** → quel processus utilise ce port (sshd, apache2, dhclient).

Utilité en troubleshooting réseau

- Vérifier si un service **écoute bien** sur le bon port (ex. serveur web sur port 80/443).
- Identifier **quel processus** utilise un port suspect.
- Détecter des **services non voulus** ou potentiellement dangereux.
- Vérifier qu'un serveur n'écoute **que sur l'IP attendue** (sécurité).

5. Connexion à distance et transfert de fichiers

Vous serez régulièrement amenés, lors de votre cursus ou dans votre vie professionnelle, à contrôler des machines à distance. Le protocole SSH est souvent utilisé pour cela, surtout dans le monde Unix.

Que fait la commande ssh ?

- **ssh** (*Secure Shell*) permet d'ouvrir une connexion sécurisée à distance entre deux machines.
- Elle fournit un **terminal interactif** sur la machine distante, comme si vous y étiez connecté directement.
- La communication est **chiffrée**, donc protégée contre les interceptions.

Comment l'utilise-t-on ?

La syntaxe de base est :

ssh utilisateur@adresse_ip

Exemple :

ssh jeremie@192.168.1.25

Qu'apporte-t-elle de plus que telnet ?

- **Chiffrement complet** des données (mot de passe, commandes, transferts).
- **Authentification par clé publique/privée** possible (plus sûr que par mot de passe).
- **Tunnels sécurisés** (redirection de ports, VPN simple).
- **Copie de fichiers sécurisée** via scp et sftp.

👉 Contrairement à **telnet**, qui envoie tout en clair (donc très peu sûr), ssh protège contre l'espionnage et l'usurpation.

Connectez-vous en ssh sur la machine de votre voisin. Vous aurez pour cela besoin de son adresse IP.

Demander son **adresse IP** (par exemple 192.168.1.42).

Utiliser :

ssh user@192.168.1.42

Saisir le mot de passe quand demandé.

👉 Si la machine de votre voisin a bien un serveur ssh actif (sshd), la connexion fonctionnera.

Utilisez **scp** ou **sftp** pour transférer un de vos fichiers « lorem ipsum » sur la machine de votre voisin.

◆ Transfert de fichiers avec scp

scp fonctionne comme **cp**, mais à travers **ssh**.

Exemple pour envoyer un fichier **lorem.txt** vers la machine du voisin :

```
scp lorem.txt user@192.168.1.42:/home/user/
```

◆ Transfert interactif avec sftp

sftp ouvre une session interactive (comme **ftp**, mais sécurisée).

Exemple :

```
sftp user@192.168.1.42
```

Puis, dans la session :

- **put lorem.txt** → envoie le fichier vers la machine distante.
- **get fichier.txt** → récupère un fichier de la machine distante.
- **ls, cd, pwd** → navigation comme en shell.

6. Fichiers de configuration Linux

Les machines Linux possèdent un système de fichiers organisé sous forme d'un arbre contenant, à la racine, un ensemble de répertoires "standards" : **/home**, **/dev**, **/mnt**, ...

Nous allons ici nous intéresser au répertoire **/etc**. L'acronyme ETC signifie **"Editable Text Configuration"**. Ce répertoire va donc contenir des fichiers de configuration au format texte, qu'il est possible d'éditer pour adapter la machine à ses besoins.

Nous allons donc nous intéresser aux fichiers contenus dans ce répertoire. Pour chaque fichier indiqué ci-dessous, consultez-le au moyen des commandes **cat**, **grep** ou **less**.

Expliquez à quoi chacun d'eux sert. Dans le cas d'un dossier, décrivez brièvement les fichiers qu'il contient. Les items marqués d'un ⚠ sont particulièrement importants pour le cours, les autres restent intéressants pour votre culture générale.

⚠ Fichiers très importants

◆ /etc/hosts

Sert à faire la correspondance entre **noms d'hôtes** et **adresses IP**, en local.

Format :

```
127.0.0.1 localhost
192.168.1.10 serveur1
```

Utile si on n'a pas de DNS ou pour forcer une résolution spécifique.

◆ /etc/resolv.conf

Contient la configuration du **DNS** (Domain Name System).

On y trouve les serveurs à interroger pour résoudre les noms de domaine en adresses IP.

Exemple :

```
nameserver 8.8.8.8
nameserver 1.1.1.1
```

◆ /etc/passwd

Contient la **liste des comptes utilisateurs** de la machine.

Chaque ligne décrit un utilisateur :

```
utilisateur:x:1000:1000:Jeremie:/home/utilisateur:/bin/bash
```

x → indique que le mot de passe est dans /etc/shadow

1000:1000 → UID et GID (identifiant utilisateur et groupe)

/home/utilisateur → dossier personnel

/bin/bash → shell par défaut

◆ /etc/shadow

Contient les **mots de passe chiffrés** des utilisateurs.

Accessible uniquement par **root**.

Exemple :

```
utilisateur:$6$QwErTy...:19422:0:99999:7:::
```

\$6\$ → algorithme de chiffrement (SHA-512 ici)

Les champs suivants donnent des infos sur la durée de validité du mot de passe.

◆ **/etc/group**

Contient la **liste des groupes** de la machine.

Format :

```
admin:x:1001:jeremie,alice
```

admin → nom du groupe

1001 → GID (identifiant du groupe)

jeremie,alice → membres du groupe

◆ **/etc/sudoers**

Définit **qui a le droit d'utiliser sudo** et comment.

⚠ **À modifier uniquement avec visudo** pour éviter les erreurs de syntaxe.

Exemple :

```
jeremie ALL=(ALL:ALL) ALL
```

→ autorise jeremie à exécuter toutes les commandes en superutilisateur.

📁 **Autres fichiers intéressants**

◆ /etc/networks

Définit des **noms symboliques** pour les réseaux.

Exemple :

```
loopback 127.0.0.0
```

```
link-local 169.254.0.0
```

◆ /etc/host.conf

Configure l'ordre de résolution des noms d'hôtes.

Exemple :

```
order hosts,bind
```

→ d'abord consulter **/etc/hosts**, puis le DNS.

◆ /etc/services

Table de correspondance entre **noms de services** et **ports TCP/UDP**.

Exemple :

```
http 80/tcp
```

```
ssh 22/tcp
```

```
dns 53/udp
```

◆ /etc/sysconfig (dossier)

Présent surtout sur **Red Hat / CentOS / Fedora**.

Contient les scripts de configuration système : réseau, firewall, clavier, etc.

◆ **/etc/default/useradd**

Définit les **valeurs par défaut** pour la création d'un utilisateur (avec useradd).

Exemple : répertoire personnel, shell par défaut, durée de validité du mot de passe.

◆ **/etc/fstab**

Liste les systèmes de fichiers à monter automatiquement au démarrage.

Exemple :

```
/dev/sda1 / ext4 defaults 0 1  
/dev/sdb1 /data ext4 defaults 0 2
```

→ permet de monter automatiquement un disque ou une partition.

◆ **/etc/rsyslog.conf**

Configuration de **rsyslog**, le système de **journalisation (logs)**.

Définit où sont envoyés les messages (fichiers locaux, serveur distant, console).

Exemple :

```
*.info;mail.none;authpriv.none;cron.none /var/log/messages
```

✓ **En résumé :**

/etc/hosts, /etc/resolv.conf → gestion des noms et DNS.

/etc/passwd, /etc/shadow, /etc/group, /etc/sudoers → gestion des utilisateurs et permissions.

/etc/fstab, /etc/services, /etc/rsyslog.conf → configuration système, montage et logs.

7. Les logs

Les logs, ou journaux système, sont les fichiers qui listent tous les événements qui se produisent dans le système.

Lorsqu'un service est bien configuré, toute action importante ou problème sera listée dans le fichier de log correspondant. Les logs sont un point d'entrée incontournable en cas de troubleshooting. Dans les systèmes **Unix/Linux**, ils sont généralement stockés dans le répertoire **/var/log**(!).

Examinez les fichiers dans **/var/log**. Essayez d'expliquer quelles informations on trouve dans chacun d'eux. Lesquels sont génériques (i.e. journaux systèmes), et lesquels sont spécifiques à un programme ou un service particulier ?

Le répertoire **/var/log** contient les **fichiers de logs** générés par le système et ses services. Quelques exemples typiques :

- **Génériques (système global)**

- **/var/log/messages** → messages système généraux (boot, réseau, erreurs, warnings).
- **/var/log/syslog** → équivalent sur **Debian/Ubuntu** (tandis que Red Hat utilise messages).
- **/var/log/dmesg** → messages du noyau (boot, détection du matériel).
- **/var/log/secure** ou **/var/log/auth.log** → authentications, sudo, SSH.

- **Spécifiques à un service ou programme**

- **/var/log/httpd/** ou **/var/log/apache2/** → logs d'Apache (accès, erreurs).
- **/var/log/mysql/** → logs de MySQL/MariaDB.
- **/var/log/cron** → tâches planifiées (cron jobs).
- **/var/log/mail.log** → serveur mail.
- **/var/log/boot.log** → détails du démarrage.

Le fichier de logs qui vous sera le plus utile est le fichier **/var/log/messages**. Comment faire pour l'examiner en temps réel ?

On utilise la commande **tail -f** :

sudo tail -f /var/log/messages

☞ Cela affiche les nouvelles lignes au fur et à mesure qu'elles sont écrites (comme un "live log").

Pour n'afficher que les 10 dernières lignes ?

On utilise simplement :

tail -n 10 /var/log/messages

☞ **-n 10** signifie : afficher les **10 dernières lignes**.

Affichez toutes les lignes du fichier **/var/log/messages** concernant le processus dhclient.

dhclient est le **client DHCP** (il demande une adresse IP au serveur DHCP).

Pour filtrer uniquement ses logs :

grep dhclient /var/log/messages

☞ Cela liste toutes les lignes où apparaît dhclient.

Si tu veux voir ces lignes **en temps réel** :

tail -f /var/log/messages | grep dhclient

8. Gestion des services (facultatif)

Un service est un programme qui tourne en arrière-plan, sous contrôle d'un gestionnaire appelé systemd dans certaines versions récentes de Linux. L'utilitaire systemctl permet de contrôler l'ensemble de ces services.

Quelle commande permet de démarrer le service SSH ?

sudo systemctl start ssh

☞ Cela **démarre** immédiatement le service **SSH** (sans l'activer au boot).

Si tu veux aussi qu'il démarre **automatiquement au démarrage** :

sudo systemctl enable ssh

Comment lister les services démarrés sur la machine ?

systemctl list-units --type=service --state=running

☞ Montre uniquement les **services actifs (running)**.

Pour voir **tous les services** (actifs, inactifs, échoués) :

systemctl list-units --type=service

A quoi correspond la notion de target dans SystemD ?

Un **target** = un **groupe d'unités/services** définissant un **état global du système**.

Exemple :

- **multi-user.target** → mode multi-utilisateur (équivalent à l'ancien runlevel 3 : console, réseau, mais pas d'interface graphique).
- **graphical.target** → mode graphique (équivalent à runlevel 5 : multi-utilisateur + interface graphique).
- **rescue.target** → mode de dépannage (équivalent runlevel 1).

☞ Donc un target regroupe plusieurs services qui doivent être actifs pour atteindre un état donné du système.

Comment afficher les services démarrés dans le target graphical ?

systemctl list-dependencies graphical.target

☞ Affiche la liste de tous les services (et cibles dépendantes) qui sont lancés pour atteindre le target graphical.

Pour n'afficher que les **services actifs** dans ce target :

systemctl list-dependencies --state=running graphical.target